

UNIVERSITÀ DI BOLOGNA



School of Engineering
Master Degree in Automation Engineering

Internship Report
**Data-Driven Model Predictive Control With Stability and
Robustness Guarantees**

Advisor:
Professor Giuseppe Notarstefano

Supervisor:
Dr.Simone Baroncini

Student: Meisam Tavakoli

Academic year 2024/2025

Abstract

This report documents the work completed during a 3-ECTS internship, focused on the implementation and study of data-driven Model Predictive Control (MPC) with stability and robustness guarantees. Based on the foundational work by J. Berberich, J. Kohler, M. A. Muller, and F. Allgower, “Data-Driven Model Predictive Control with Stability and Robustness Guarantees”, the internship aimed at deepening understanding of behavioral systems theory and its application to MPC through hands-on coding in Python using CasADi. No original research was conducted; instead, the internship focused on reproducing and understanding the theoretical and algorithmic components of the cited work.

Contents

1	Introduction	4
2	Internship Objectives	4
3	Theoretical Background	5
3.1	Behavioral Systems Perspective	5
3.2	Hankel Matrix Construction	5
3.3	Data-Driven MPC Formulations	5
3.4	Behavioral Systems Theory	5
4	Tools and System Setup	6
4.1	Software Stack	6
4.2	Benchmark System: The Four-Tank Process	6
4.3	Initial Data Collection	7
5	Implementation	8
5.1	Hankel Matrix Construction	8
5.2	Optimization Variables and Cost Function	9
5.3	Constraints	9
5.4	CasADi Implementation	10
6	Implementation	10
6.1	Hankel Matrix Construction	11
6.2	Cost Function and Constraints Overview	11
7	Simulation and Results	12
7.1	Simulation Setup	12
7.2	Nominal MPC Results	13
7.3	Robust MPC Results	18
7.4	Robust MPC Results	18
7.5	Solver Performance Analysis	22
8	Conclusion and Future Work	23

1 Introduction

Model Predictive Control (MPC) is a widely used optimal control technique known for its ability to handle multivariable systems and constraints systematically. At each time step, MPC solves a finite-horizon optimization problem to compute control actions, which are then applied in a receding horizon fashion. However, one of the fundamental limitations of conventional MPC approaches is their dependence on accurate mathematical models of the underlying system dynamics. Obtaining such models through first-principles derivation or classical system identification techniques can be time-consuming, error-prone, and infeasible for complex or poorly understood systems.

To address these challenges, recent research has shifted toward data-driven control approaches, which aim to leverage measured data directly for control design, bypassing the explicit modeling step. Among these, a particularly promising framework is the one introduced by [1], which formulates an MPC scheme based purely on measured input-output trajectories without relying on a parametric model.

This data-driven MPC framework utilizes concepts from behavioral systems theory, particularly Willems' Fundamental Lemma, to express system trajectories as linear combinations of past data sequences. Under the assumption that the input is persistently exciting, it is possible to reconstruct future behavior of the system directly from historical data using Hankel matrices. This eliminates the need for an intermediate model identification step and enables the implementation of MPC with theoretical guarantees on stability and robustness.

The objective of this internship was to gain practical understanding of such data-driven control methods through a structured implementation of the aforementioned MPC framework. The internship involved in-depth exploration of the theoretical foundations, careful replication of the proposed algorithms using Python and CasADi, and evaluation of the control performance on a benchmark four-tank system. Rather than contributing novel research, the focus was on accurately reproducing and understanding the work of [1], thereby solidifying core competencies in advanced control and optimization.

In the following sections, we present the internship objectives, the theoretical background of the method, details of the implementation, simulation results, challenges encountered, and concluding remarks on the learning outcomes and potential future work.

2 Internship Objectives

The primary aim of this internship was to gain hands-on experience with modern control methodologies that leverage data rather than explicit mathematical models. The tasks were designed to develop both theoretical understanding and practical implementation skills aligned with the framework introduced by [1]. The internship comprised the following structured objectives:

1. **Familiarization with CasADi for Optimization:** Gain proficiency in using CasADi, a symbolic framework for numerical optimization in Python, with a specific focus on constructing and solving nonlinear and quadratic programming problems relevant to MPC.
2. **System Identification Using Data-Driven Methods:** Implement data acquisition routines to generate persistently exciting input-output trajectories. Use these trajectories to construct Hankel matrices as required by the behavioral systems theory framework.
3. **Implementation of Data-Driven MPC Algorithms:** Reproduce the nominal and robust data-driven MPC schemes described by Berberich et al., including terminal constraints and the handling of bounded measurement noise through slack variables and regularization terms.
4. **Simulation on a Benchmark System:** Simulate the implemented controllers on a discretized linear model of the four-tank system. This includes applying the controller in a closed-loop setting and analyzing its ability to track reference outputs and maintain robustness against disturbances.

Each objective was pursued in a step-by-step manner, reinforcing both theoretical knowledge and implementation capability. The following sections of this report detail the methods used and the insights gained in the course of meeting these objectives.

3 Theoretical Background

This section presents the theoretical underpinnings of the data-driven MPC scheme explored during the internship. All material in this section is directly grounded in the work of [1] and [2] with no extrapolations or modifications.

3.1 Behavioral Systems Perspective

Traditional control methods typically rely on an explicit model of the system dynamics, either derived from first principles or obtained through parameter estimation. The data-driven framework employed in this internship takes a fundamentally different perspective based on behavioral systems theory. Rather than modeling system states or parameters, it represents the set of all possible input-output trajectories as a data-driven linear space.

A key result from this theory is that if the input trajectory used to collect data is persistently exciting of a sufficiently high order, then all future system trajectories can be represented as linear combinations of measured data. This result enables a model-free representation of the system suitable for prediction and control.

3.2 Hankel Matrix Construction

Given an input-output trajectory

$$\{u_k, y_k\}_{k=0}^{N-1}$$

from an unknown discrete-time linear time-invariant (LTI) system, the approach constructs a block Hankel matrix of the form:

$$H_L(u_d, y_d) = \begin{bmatrix} H_L(u_d) \\ H_L(y_d) \end{bmatrix}, \quad (1)$$

where $H_L(u_d)$ and $H_L(y_d)$ are the Hankel matrices of the input and output trajectories, respectively. These matrices span the trajectory space of the system and serve as a foundation for control synthesis

3.3 Data-Driven MPC Formulations

Using the Hankel-based representation, Berberich et al. formulate two variations of the data-driven MPC problem:

- **Nominal MPC:** Assumes noise-free measurements. Future trajectories are predicted using a linear combination of data-driven trajectories, and terminal equality constraints are included to guarantee exponential stability.
- **Robust MPC:** Accounts for bounded additive measurement noise. A slack variable and corresponding regularization terms are introduced in the cost function, and robustness constraints are imposed to maintain feasibility.

Both schemes use a receding horizon strategy and optimize over trajectory weights rather than control inputs directly. These weights are used to reconstruct input-output trajectories from the Hankel matrix.

The upcoming sections detail how these theoretical constructs were implemented in code and validated through simulation.

3.4 Behavioral Systems Theory

[1] uses behavioral systems theory to avoid explicit model identification. A key result is that a Hankel matrix constructed from a persistently exciting input-output trajectory can represent all future trajectories of the system.

4 Tools and System Setup

This section introduces the software tools used during the internship and the benchmark system model employed for implementation and validation.

4.1 Software Stack

The implementation was carried out entirely in Python, using the following libraries:

- **CasADi:** A symbolic framework for automatic differentiation and numerical optimization [4]. CasADi was used to define the MPC optimization problem, construct cost functions, impose constraints, and generate efficient solver code.
- **NumPy and SciPy:** These were used for numerical computations, matrix operations, and signal generation (e.g., autocorrelation analysis).
- **Matplotlib:** Employed for data visualization, including input-output trajectories and system responses.

4.2 Benchmark System: The Four-Tank Process

The four-tank system is a well-known multivariable control benchmark with two inputs and two outputs. For this internship, a discrete-time linearized model of the system was used, given by:

$$x_{k+1} = Ax_k + Bu_k, \quad (2)$$

$$y_k = Cx_k + Du_k, \quad (3)$$

where:

- $x_k \in \mathbb{R}^4$ is the state vector (water levels in the tanks),
- $u_k \in \mathbb{R}^2$ is the input vector (pump voltages),
- $y_k \in \mathbb{R}^2$ is the output vector (measured levels),
- $A \in \mathbb{R}^{4 \times 4}, B \in \mathbb{R}^{4 \times 2}, C \in \mathbb{R}^{2 \times 4}, D \in \mathbb{R}^{2 \times 2}$ are system matrices derived from linearization.

The specific matrices used in the implementation are:

$$A = \begin{bmatrix} 0.921 & 0 & 0.041 & 0 \\ 0 & 0.918 & 0 & 0.033 \\ 0 & 0 & 0.924 & 0 \\ 0 & 0 & 0 & 0.937 \end{bmatrix}, \quad B = \begin{bmatrix} 0.017 & 0.001 \\ 0.001 & 0.023 \\ 0 & 0.061 \\ 0.072 & 0 \end{bmatrix},$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

The model was implemented in CasADi using symbolic variables and functions to enable efficient numerical simulation and optimization. Discretization was assumed to be consistent with the sampling rate used during the data collection and MPC execution.

4.3 Initial Data Collection

Prior to controller synthesis, input-output data was generated using a pseudo-random binary signal (PRBS) to ensure persistent excitation. A total of $N = 420$ samples were collected. The resulting trajectories were used to construct Hankel matrices required for data-driven prediction.

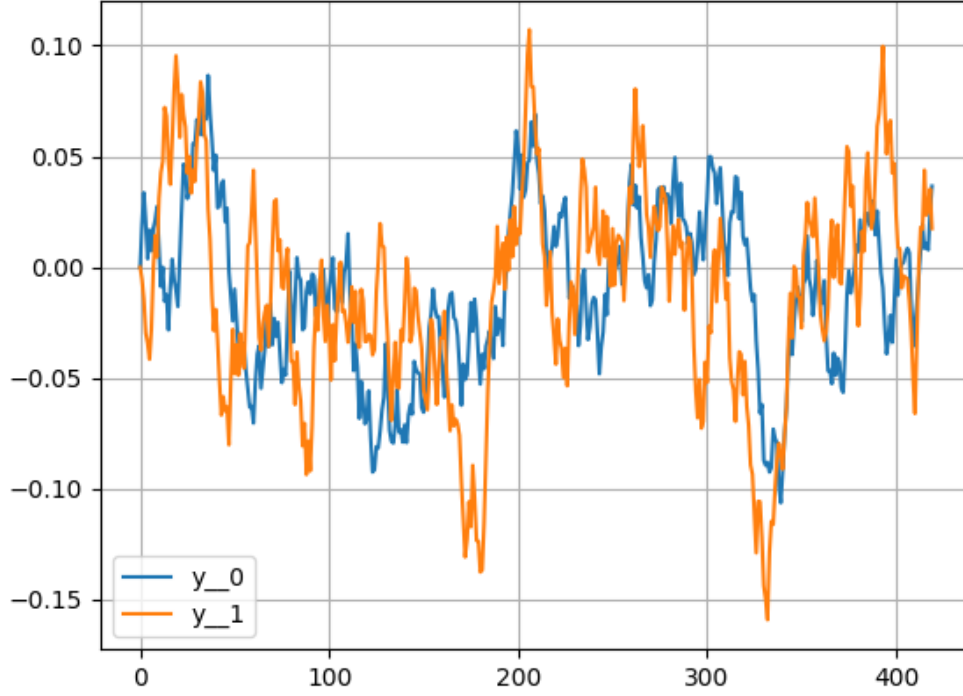


Figure 1: Historical output data collected from the four-tank system. The plot shows two output channels, y_0 and y_1 , corresponding to the water levels in the two lower tanks, which are the measurable outputs of the system.

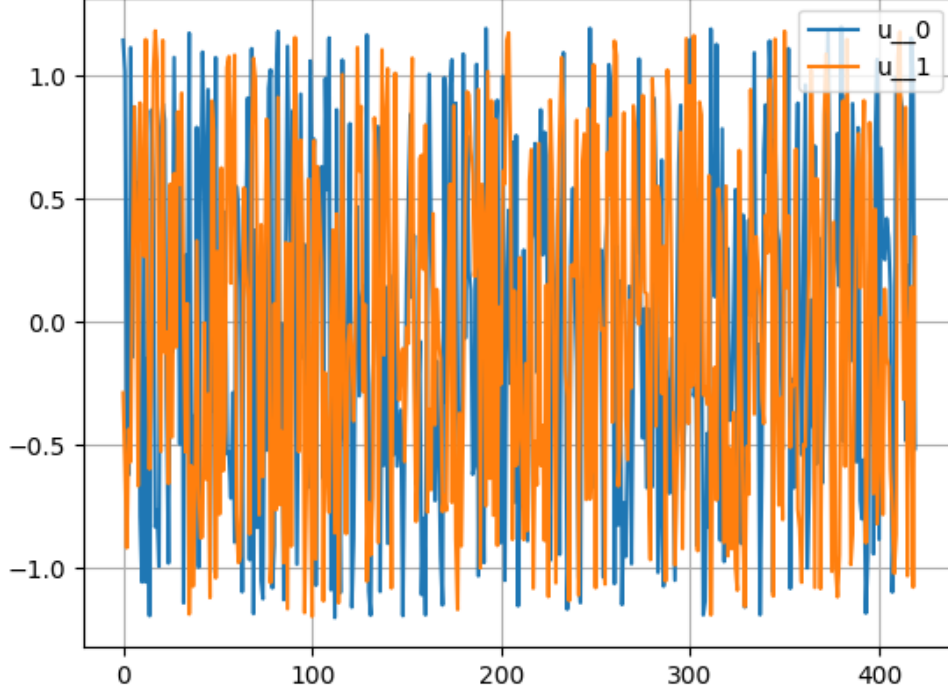


Figure 2: PRBS-driven historical input signals used for excitation. The system has two control inputs, u_0 and u_1 , which represent the voltages applied to the two pumps. These were designed to be persistently exciting to satisfy the requirements of the behavioral approach.

Input and output signals were constrained according to actuator and sensor limitations. Optional noise was added to outputs to test the robustness of the controller.

The next section details how the controller was implemented, including the construction of the optimization problem and its robust variant.

5 Implementation

This section presents the detailed implementation of the data-driven MPC algorithms, with emphasis on mathematical formulation, symbolic modeling in CasADi, and robustification mechanisms.

5.1 Hankel Matrix Construction

As discussed earlier, a Hankel matrix is constructed from the historical input-output data:

$$H_L(u_d, y_d) = \begin{bmatrix} H_L(u_d) \\ H_L(y_d) \end{bmatrix} \in \mathbb{R}^{(m+p)L \times (T-L+1)} \quad (4)$$

where:

- $H_L(u_d) \in \mathbb{R}^{mL \times (T-L+1)}$ is the input Hankel matrix,
- $H_L(y_d) \in \mathbb{R}^{pL \times (T-L+1)}$ is the output Hankel matrix,
- $L = T_{ini} + T_{pred}$ is the total horizon (initial condition and prediction windows).

The collected trajectory is persistently exciting of order $L + n$ if the rank condition:

$$\text{rank}(H_{L+n}(u_d)) = m(L + n) \quad (5)$$

is satisfied. This ensures that all future trajectories can be expressed using the data-driven basis.

5.2 Optimization Variables and Cost Function

In the nominal data-driven MPC formulation, the optimization problem is defined over three decision variables:

- $\alpha(t) \in \mathbb{R}^{N-L+1}$: the coefficient vector used to reconstruct trajectories from data,
- $\bar{u}(t) \in \mathbb{R}^{mL}$: the predicted future input trajectory,
- $\bar{y}(t) \in \mathbb{R}^{pL}$: the predicted future output trajectory.

The cost function is defined as:

$$J_L^*(u_{[t-n, t-1]}, y_{[t-n, t-1]}) = \min_{\alpha(t), \bar{u}(t), \bar{y}(t)} \sum_{k=0}^{L-1} \ell(\bar{u}_k(t), \bar{y}_k(t)) \quad (6)$$

subject to the following constraints:

$$\begin{bmatrix} \bar{u}_{[-n, L-1]}(t) \\ \bar{y}_{[-n, L-1]}(t) \end{bmatrix} = \begin{bmatrix} H_{L+n}(u^d) \\ H_{L+n}(y^d) \end{bmatrix} \alpha(t) \quad (3b)$$

$$\begin{bmatrix} \bar{u}_{[-n, -1]}(t) \\ \bar{y}_{[-n, -1]}(t) \end{bmatrix} = \begin{bmatrix} u_{[t-n, t-1]} \\ y_{[t-n, t-1]} \end{bmatrix} \quad (3c)$$

$$\begin{bmatrix} \bar{u}_{[L-n, L-1]}(t) \\ \bar{y}_{[L-n, L-1]}(t) \end{bmatrix} = \begin{bmatrix} u_n^s \\ y_n^s \end{bmatrix} \quad (3d)$$

and the predicted trajectories must satisfy input/output constraints:

$$\bar{u}_k(t) \in \mathcal{U}, \quad \bar{y}_k(t) \in \mathcal{Y}, \quad k \in \mathbb{I}_{[0, L-1]} \quad (3e)$$

This formulation ensures that future behavior is constrained to lie in the span of past data, initialized with current measurements, and guided to converge toward a desired steady-state operating point.

5.3 Constraints

Now we describe the concept of each constraint that defines the feasible set of the optimization problem. These constraints ensure that the predicted trajectories generated by the controller are consistent with the system's behavior, initialized correctly, and driven toward a desired equilibrium.

- **Data consistency constraint:**

$$\begin{bmatrix} \bar{u}_{[-n, L-1]}(t) \\ \bar{y}_{[-n, L-1]}(t) \end{bmatrix} = \begin{bmatrix} H_{L+n}(u^d) \\ H_{L+n}(y^d) \end{bmatrix} \alpha(t)$$

This enforces that the predicted input-output sequence lies in the space spanned by previously measured trajectories, consistent with the behavioral framework.

- **Initial condition constraint:**

$$\begin{bmatrix} \bar{u}_{[-n, -1]}(t) \\ \bar{y}_{[-n, -1]}(t) \end{bmatrix} = \begin{bmatrix} u_{[t-n, t-1]} \\ y_{[t-n, t-1]} \end{bmatrix}$$

This ensures that the predicted trajectory starts from the most recent system measurements and follows a physically plausible path.

- **Terminal constraint:**

$$\begin{bmatrix} \bar{u}_{[L-n, L-1]}(t) \\ \bar{y}_{[L-n, L-1]}(t) \end{bmatrix} = \begin{bmatrix} u_n^s \\ y_n^s \end{bmatrix}$$

This constraint drives the last part of the predicted trajectory to match a known steady-state input-output pair, which is essential for guaranteeing closed-loop exponential stability.

- **Input and output bounds:**

$$\bar{u}_k(t) \in \mathcal{U}, \quad \bar{y}_k(t) \in \mathcal{Y}, \quad \forall k \in \mathbb{I}_{[0, L-1]}$$

These constraints ensure that the applied inputs and resulting outputs remain within physical or operational bounds imposed by the system.

- **Robustness constraint (robust formulation only):**

$$\|\sigma_k\|_\infty \leq \epsilon \cdot (1 + \|\alpha(t)\|_1), \quad \forall k \in \mathbb{I}_{[0, L-1]}$$

This constraint limits the effect of measurement noise by bounding the slack variable σ in terms of the magnitude of $\alpha(t)$, as proposed in [2].

5.4 CasADi Implementation

All decision variables, constraints, and cost functions were symbolically modeled using CasADi's **SX** framework [4]. The optimization problem was compiled and solved using either IPOPT (nonlinear) or OSQP (quadratic) solvers.

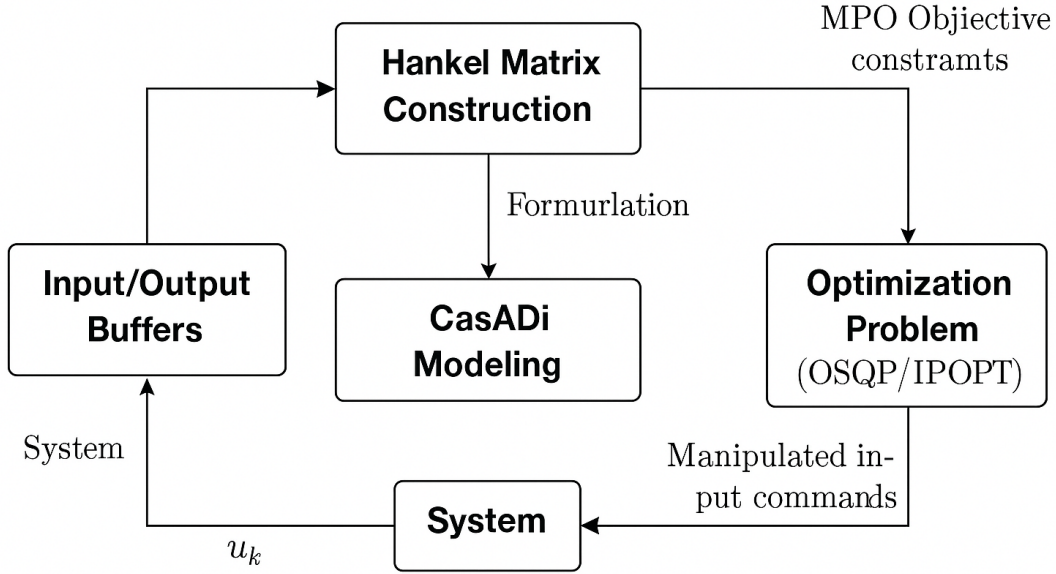


Figure 3: Schematic structure of the data-driven MPC solver created using CasADi.

In the next section, we present the results of closed-loop simulations conducted using both the nominal and robust formulations.

6 Implementation

This section presents the detailed implementation of the data-driven MPC algorithms.

6.1 Hankel Matrix Construction

The data-driven MPC framework leverages Hankel matrices to encode dynamic information from measured input-output trajectories. Rather than estimating a parametric model, the approach relies on behavioral system theory, where future trajectories are represented as linear combinations of previously observed data.

Let $\{u_k, y_k\}_{k=0}^{T-1}$ be a measured trajectory with $u_k \in \mathbb{R}^m$ and $y_k \in \mathbb{R}^p$. For a given horizon length L , the block Hankel matrices of depth L are constructed as:

$$\mathcal{H}_L(u) := \begin{bmatrix} u_0 & u_1 & \cdots & u_{T-L} \\ u_1 & u_2 & \cdots & u_{T-L+1} \\ \vdots & \vdots & \ddots & \vdots \\ u_{L-1} & u_L & \cdots & u_{T-1} \end{bmatrix} \in \mathbb{R}^{mL \times (T-L+1)} \quad (7)$$

$$\mathcal{H}_L(y) := \begin{bmatrix} y_0 & y_1 & \cdots & y_{T-L} \\ y_1 & y_2 & \cdots & y_{T-L+1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{L-1} & y_L & \cdots & y_{T-1} \end{bmatrix} \in \mathbb{R}^{pL \times (T-L+1)} \quad (8)$$

These matrices are horizontally stacked to form the combined Hankel matrix:

$$\mathcal{H}_L(u, y) := \begin{bmatrix} \mathcal{H}_L(u) \\ \mathcal{H}_L(y) \end{bmatrix} \quad (9)$$

To enable prediction and control over the system, the collected input signal must be persistently exciting of order $L + n$, where n is the assumed system order. This ensures that the Hankel matrix has full row rank and spans the space of admissible trajectories.

The Hankel matrix is partitioned into:

- **Past** window of length T_{ini} , representing initial conditions
- **Future** window of length T_f , used for optimization

The resulting formulation underpins the optimization problem solved in each MPC cycle, allowing trajectory prediction via data alone.

6.2 Cost Function and Constraints Overview

In data-driven MPC, the control task is posed as a constrained optimization problem over future input-output trajectories, which are implicitly defined by the Hankel data and a coefficient vector α . Rather than optimizing over control inputs directly, the optimizer solves for α such that the predicted trajectory matches desired behavior.

Let T_f denote the prediction horizon. The predicted trajectory is constructed as:

$$\begin{bmatrix} u_{\text{ini}} \\ y_{\text{ini}} \\ u_{\text{pred}} \\ y_{\text{pred}} \end{bmatrix} = \mathcal{H}_{T_{\text{ini}}+T_f}(u, y) \cdot \alpha \quad (10)$$

where $u_{\text{ini}}, y_{\text{ini}}$ are the known recent measurements (initial window), and $u_{\text{pred}}, y_{\text{pred}}$ are the predicted control inputs and outputs. The vector α contains the coefficients of the linear combination of trajectories from the data.

The cost function penalizes tracking error and control effort over the prediction horizon:

$$J(\alpha) = \sum_{k=0}^{T_f-1} (\|y_k - y_{\text{ref}}\|_Q^2 + \|u_k - u_{\text{ref}}\|_R^2) \quad (11)$$

where $Q \succeq 0$ and $R \succ 0$ are tuning matrices, and $y_{\text{ref}}, u_{\text{ref}}$ are reference targets.

In the robust formulation, a slack variable σ is introduced to account for bounded measurement noise, leading to a modified cost:

$$J_{\text{robust}}(\alpha, \sigma) = J(\alpha) + \lambda_g \|\alpha\|_2^2 + \lambda_\sigma \|\sigma\|_2^2 \quad (12)$$

The optimization is constrained by:

- **Data constraint:** the predicted trajectory must lie in the column space of the Hankel matrix.
- **Initial condition consistency:** the initial T_{ini} steps must match measured inputs and outputs.
- **Terminal constraint:** the final segment of the trajectory should match the reference to ensure stability.
- **Input and output bounds:** $u_k \in \mathcal{U}, y_k \in \mathcal{Y}$ for all k .
- **Robustness constraint:** $\|\sigma_k\|_\infty \leq \epsilon(1 + \|\alpha\|_1)$ at each time step.

This formulation encapsulates the predictive control task using only data, without identifying a parametric model. In the next phase, this is implemented symbolically and solved at each control interval.

Implementation Summary

The data-driven MPC controller was implemented using a modular design [11], enabling flexibility between nominal and robust formulations. Each component, from Hankel matrix assembly to optimization, was embedded symbolically using CasADi, ensuring compatibility with both linear and nonlinear extensions. The optimization problem was solved using OSQP due to its speed and warm-starting capability, but the framework is general enough to accommodate solvers like IPOPT if required.

The formulations presented rely exclusively on measured data, circumventing the need for system identification. Importantly, the theoretical guarantees, including closed-loop stability and robust constraint satisfaction [2], were respected through the careful enforcement of terminal constraints, regularization, and persistency of excitation. This framework sets the foundation for the simulation and analysis of both nominal and noise-affected scenarios in the next section.

7 Simulation and Results

This section presents the results of closed-loop simulations conducted using both the nominal and robust data-driven MPC controllers. The goal is to validate the theoretical guarantees in practice and highlight the difference in behavior under measurement noise.

7.1 Simulation Setup

The test system is the discrete-time linearized four-tank process introduced earlier. The simulation is conducted over 180 MPC cycles with a prediction horizon of $T_f = 28$ and an initial condition window $T_{\text{ini}} = 4$, yielding a total horizon length of $L = T_{\text{ini}} + T_f = 32$.

The input and output reference values, u_{ref} and y_{ref} , were chosen to correspond to the system's steady-state configuration. The cost function weights were selected as:

$$Q = 2.8 \cdot I_2,$$

$$R = 1.2 \cdot 10^{-4} \cdot I_2$$

where I_2 is the 2×2 identity matrix, consistent with the system's two output and two input channels, to penalize output deviation and input usage, respectively. Input bounds were enforced at ± 9.5 , while output constraints were left inactive in the nominal case.

To ensure data richness, the Hankel matrix was built from $N = 420$ samples of PRBS-driven input-output data. The PRBS signals were scaled and validated for persistency of excitation by confirming the rank condition:

$$\text{rank}(\mathcal{H}_{L+n}(u)) = m(L+n) \quad (13)$$

Each simulation consisted of three phases:

1. **Initialization:** Build Hankel matrix using collected data and define solver settings.
2. **Execution:** At each step, solve the optimization problem using current measurements and apply the first input from the optimal predicted sequence.
3. **Update:** Shift the measurement window and warm-start the solver with previous values.

In the robust case, additive measurement noise was applied to the output signals, bounded by:

$$\|\varepsilon_k\|_\infty \leq \epsilon = 1.8 \times 10^{-3} \quad (14)$$

Regularization weights were tuned to:

$$\lambda_g = 0.12/\epsilon,$$

$$\lambda_\sigma = 1.2 \times 10^3$$

to maintain feasibility and mitigate noise effects.

7.2 Nominal MPC Results

The first scenario considers the idealized case in which the collected and real-time measurement data are noise-free. The nominal data-driven MPC problem follows the structure of Problem (3) in [2] and relies on a single measured trajectory for system parametrization.

At each time step t , the optimization problem solved follows the nominal data-driven MPC formulation:

$$\min_{\alpha(t), \bar{u}(t), \bar{y}(t)} \sum_{k=0}^{L-1} (\|\bar{u}_k(t) - u_{\text{ref}}\|_R^2 + \|\bar{y}_k(t) - y_{\text{ref}}\|_Q^2) \quad (15)$$

subject to the following constraints:

$$\begin{bmatrix} \bar{u}_{[-n, L-1]}(t) \\ \bar{y}_{[-n, L-1]}(t) \end{bmatrix} = \begin{bmatrix} \mathcal{H}_{L+n}(u^d) \\ \mathcal{H}_{L+n}(y^d) \end{bmatrix} \alpha(t) \quad (16)$$

$$\begin{bmatrix} \bar{u}_{[-n, -1]}(t) \\ \bar{y}_{[-n, -1]}(t) \end{bmatrix} = \begin{bmatrix} u_{[t-n, t-1]} \\ y_{[t-n, t-1]} \end{bmatrix} \quad (17)$$

$$\begin{bmatrix} \bar{u}_{[L-n, L-1]}(t) \\ \bar{y}_{[L-n, L-1]}(t) \end{bmatrix} = \begin{bmatrix} u_n^s \\ y_n^s \end{bmatrix} \quad (18)$$

The predicted trajectories $\bar{u}_k(t) \in \mathcal{U}$, $\bar{y}_k(t) \in \mathcal{Y}$ for all $k \in \mathbb{I}_{[0, L-1]}$ must also satisfy input and output bounds, and the structure of the terminal constraint ensures convergence toward the desired steady-state reference.

Constraints (16) and (17) ensure that the predicted trajectories are both consistent with the behavioral model and initialized with actual measurements. The terminal constraint (18), imposed over n time steps, ensures exponential convergence to the setpoint.

In simulation, the controller was applied over 180 cycles. Output signals converged to their references rapidly and without oscillations, validating the nominal DDMPC framework's ability to stabilize the system using only data.

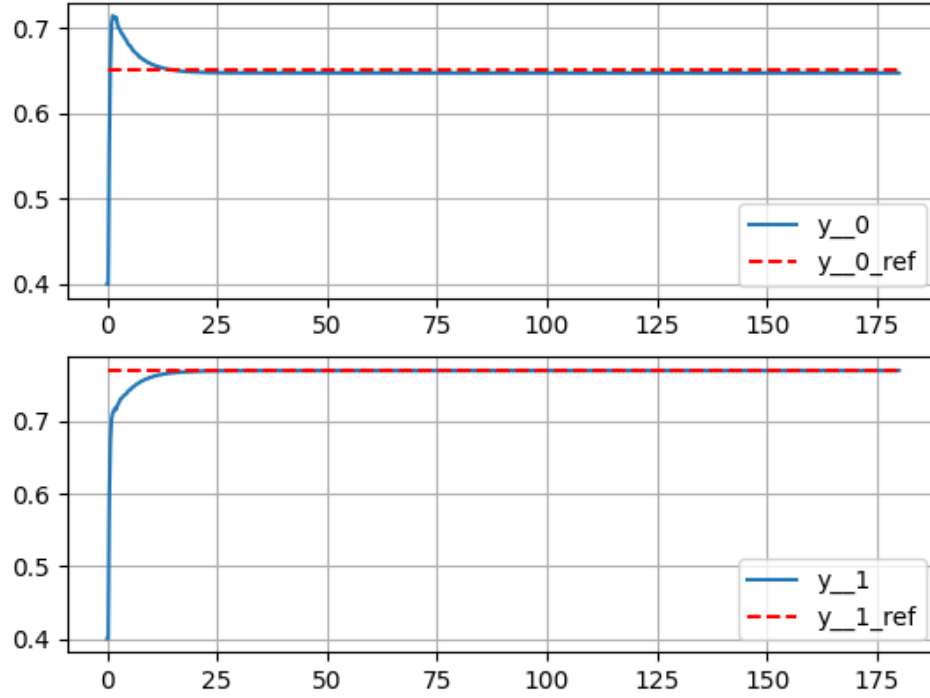


Figure 4: Closed-loop output trajectories under nominal DDMPC. Outputs y_0 and y_1 converge smoothly to their references.

The control inputs generated were within physical bounds and exhibited minimal actuation variation, confirming proper regularization and numerical stability.

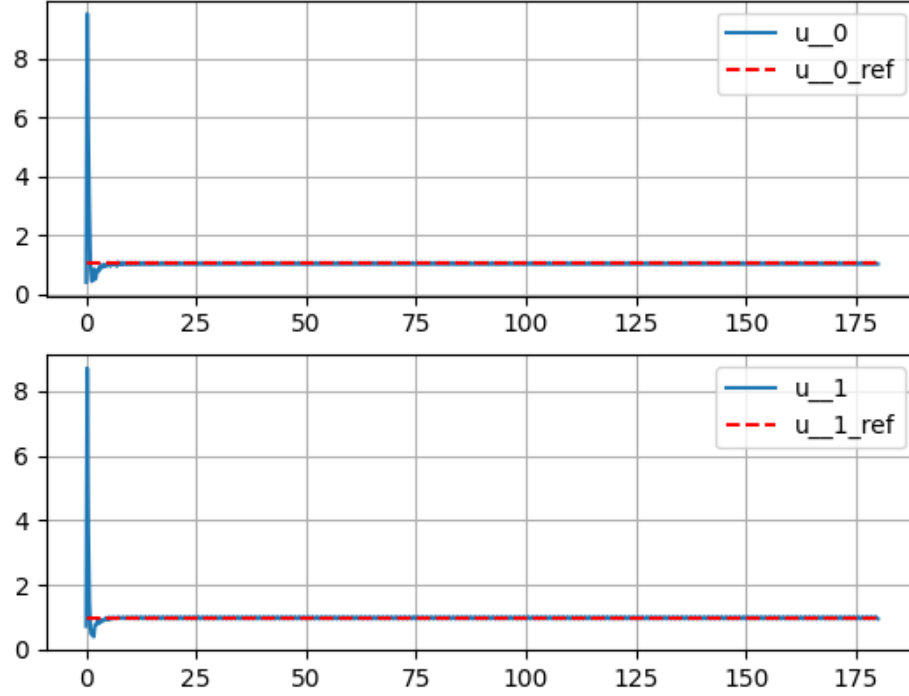


Figure 5: Control inputs u_0 and u_1 applied by the nominal controller. No saturation occurred.

The average solver time per cycle remained below 10 ms, allowing the method to be used in real-time settings with moderate update rates.

This progressive convergence is visually apparent throughout the control execution. At each iteration, the input and output signals increasingly align with their references. This effect arises from the receding horizon nature of MPC, where the optimization problem is solved repeatedly using updated measurements and warm-started solutions. In particular:

- The outputs y_0 , y_1 steadily reduce their deviation from y_{ref} across iterations.
- The control inputs u_0 , u_1 converge toward their optimal steady-state values u_{ref} , while remaining within bounds.

These iterative plots provide both diagnostic insight and validation of the algorithm's behavior. They serve as a form of visual Lyapunov function: each successive curve demonstrates reduced tracking error and smoothed control effort.

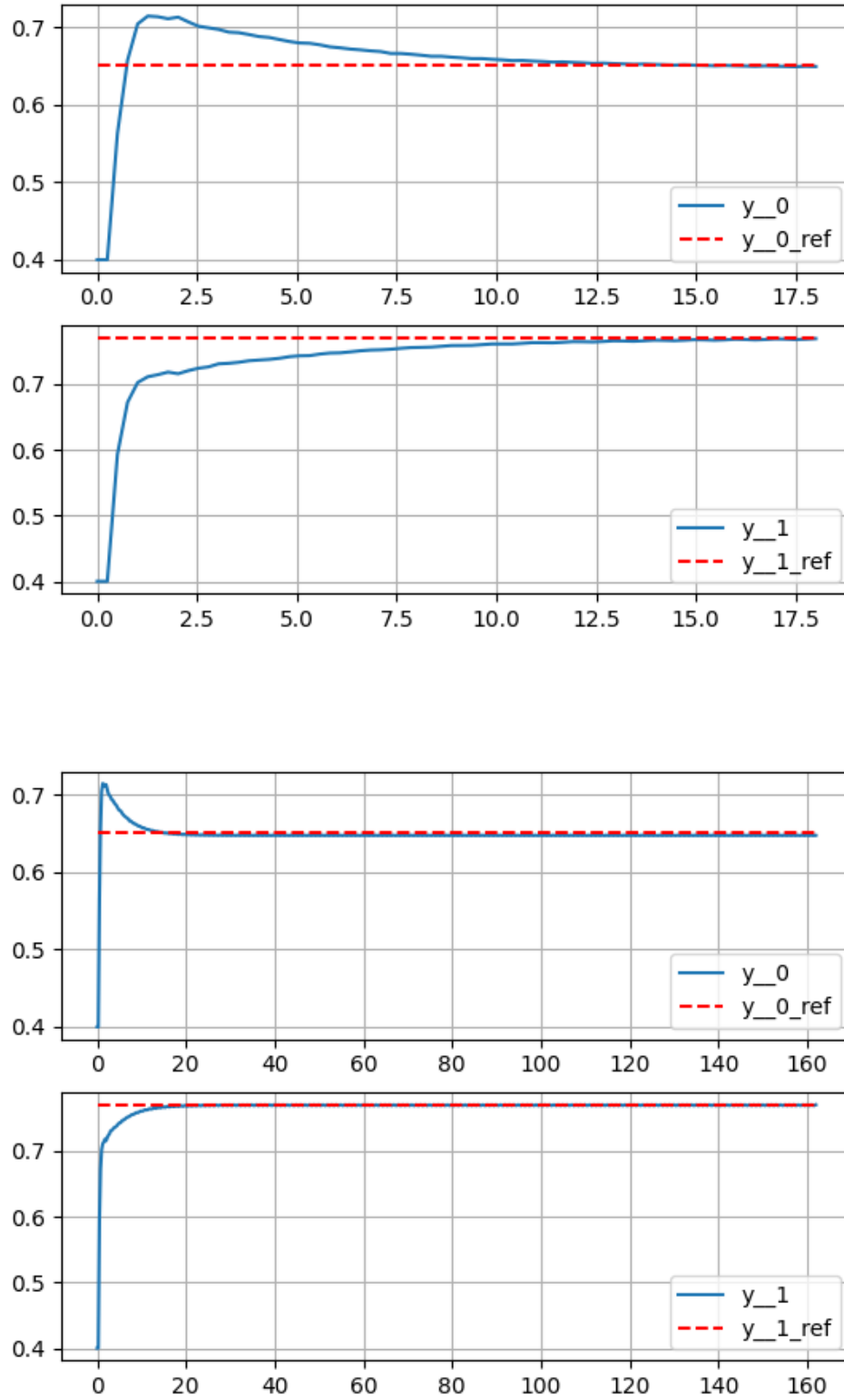


Figure 6: Example of output convergence: y_0 and y_1 approach their references over multiple control iterations.

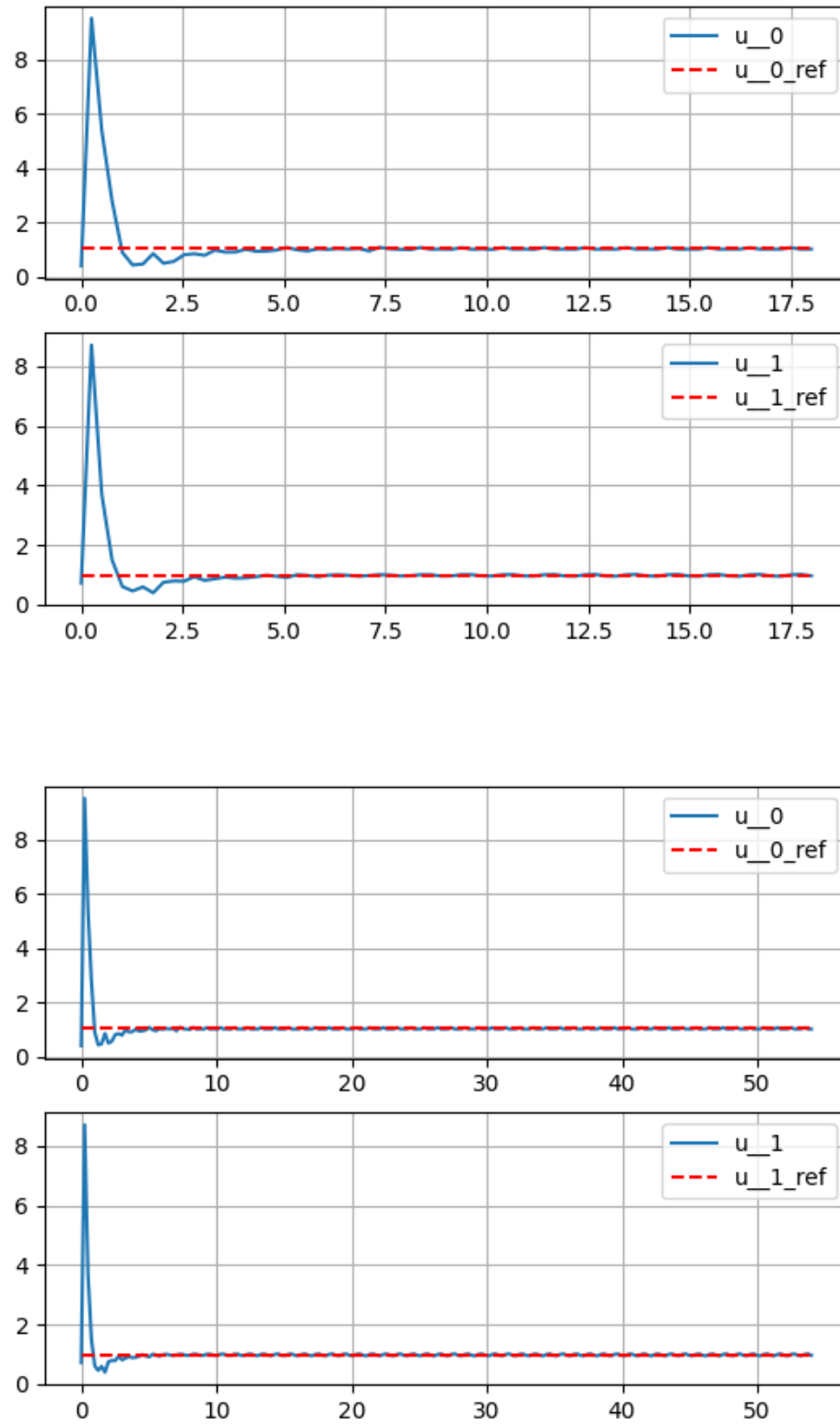


Figure 7: Input convergence of u_0 and u_1 toward steady-state values. Oscillations reduce over time.

7.3 Robust MPC Results

In practical settings, output measurements are often corrupted by bounded noise, which, if not properly addressed, can lead to infeasibility or degraded tracking performance in nominal DDMPC. To account for this, the controller was reconfigured using the robust formulation described in [2, 3], based on Problem (4) in [2].

7.4 Robust MPC Results

To address bounded measurement noise, the robust data-driven MPC formulation includes an additional slack variable $\sigma(t)$ and modifies the cost and constraints accordingly. This allows the controller to maintain feasibility and stability guarantees even when output data is perturbed.

The optimization problem at time t is given by:

$$\min_{\alpha(t), \bar{u}(t), \bar{y}(t), \sigma(t)} \sum_{k=0}^{L-1} \ell(\bar{u}_k(t), \bar{y}_k(t)) + \lambda_\alpha \bar{\epsilon} \|\alpha(t)\|_2^2 + \lambda_\sigma \|\sigma(t)\|_2^2 \quad (19)$$

subject to the constraints:

$$\begin{bmatrix} \bar{u}(t) \\ \bar{y}(t) + \sigma(t) \end{bmatrix} = \begin{bmatrix} H_{L+n}(u^d) \\ H_{L+n}(y^d) \end{bmatrix} \alpha(t) \quad (4b)$$

$$\begin{bmatrix} \bar{u}_{[-n,-1]}(t) \\ \bar{y}_{[-n,-1]}(t) \end{bmatrix} = \begin{bmatrix} u_{[t-n,t-1]} \\ y_{[t-n,t-1]} \end{bmatrix} \quad (4c)$$

$$\begin{bmatrix} \bar{u}_{[L-n,L-1]}(t) \\ \bar{y}_{[L-n,L-1]}(t) \end{bmatrix} = \begin{bmatrix} u_n^s \\ y_n^s \end{bmatrix}, \quad \bar{u}_k(t) \in \mathcal{U} \quad (4d)$$

and the noise variable is constrained by:

$$\|\sigma_k(t)\|_\infty \leq \bar{\epsilon} \cdot (1 + \|\alpha(t)\|_1), \quad \forall k \in \mathbb{I}_{[0,L-1]} \quad (4e)$$

This structure ensures that the controller can accommodate bounded disturbances while preserving the convergence and constraint satisfaction guarantees. The regularization weights λ_α and λ_σ penalize complexity in trajectory selection and noise deviation, respectively.

The regularization terms in the cost penalize overly flexible reconstructions of the trajectory that might otherwise fit noise. As shown in [2], this yields practical exponential stability, meaning the system converges to a small neighborhood around the reference depending on the noise level ϵ .

Simulations were conducted using the same prediction and initialization horizons as in the nominal case. Measurement noise was injected as bounded additive perturbations:

$$\|\varepsilon_k\|_\infty \leq \epsilon = 1.8 \times 10^{-3}$$

and the regularization weights were tuned as:

$$\lambda_\alpha = \frac{0.12}{\epsilon}, \quad \lambda_\sigma = 1.2 \times 10^3$$

The closed-loop response confirms robust performance: the outputs remained close to reference values, with tracking error bounded as expected.

As in the nominal case, the evolution of input and output measurements across control iterations revealed consistent improvements in tracking. Despite the presence of measurement noise, each successive plot shows a tightening of the response around the reference. This reflects the robust controller's ability to attenuate disturbances and converge within a guaranteed bound.

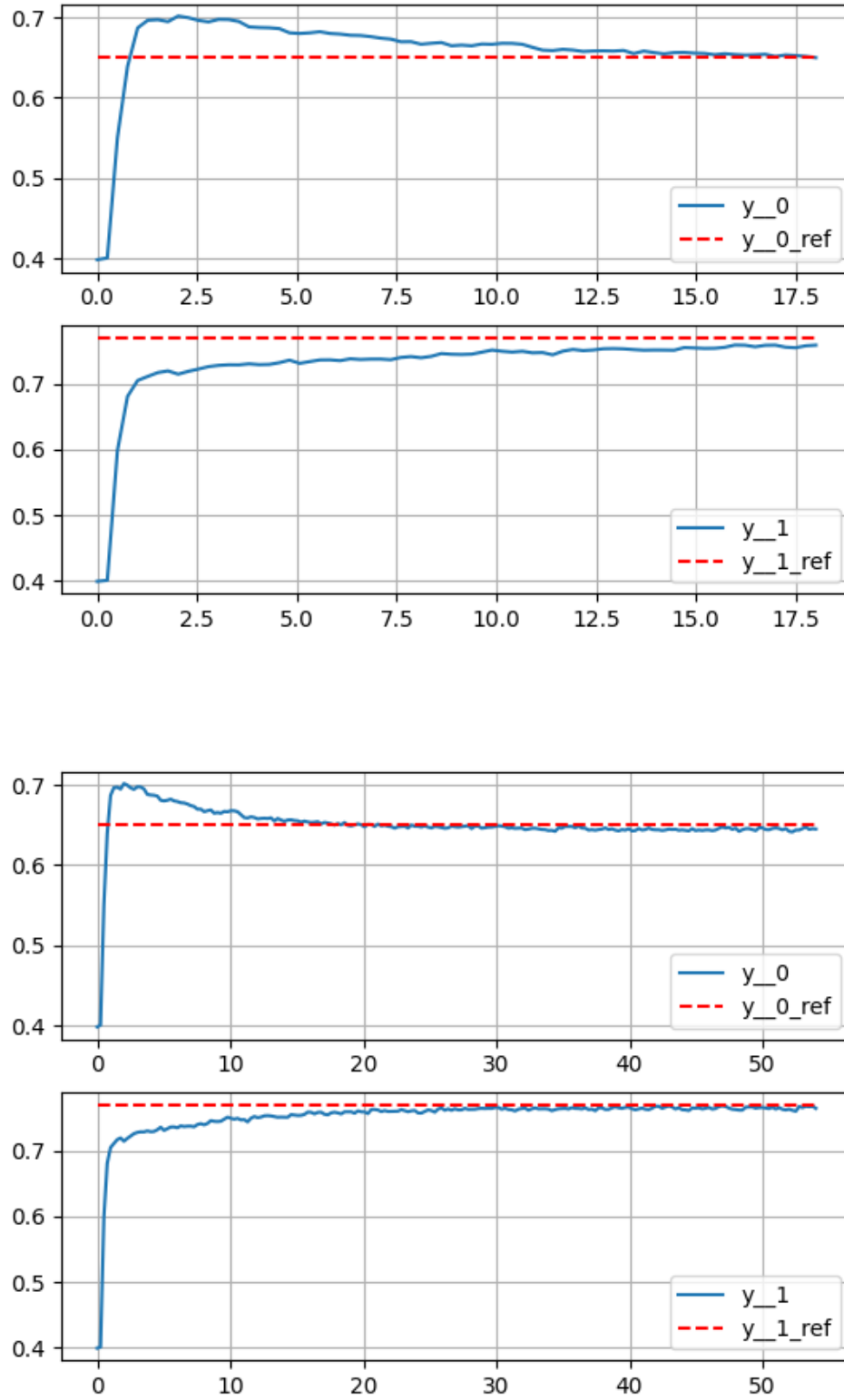


Figure 8: Intermediate output trajectories under robust DDMPC. Outputs fluctuate slightly but steadily contract toward y_{ref} . (Insert plot series here.)

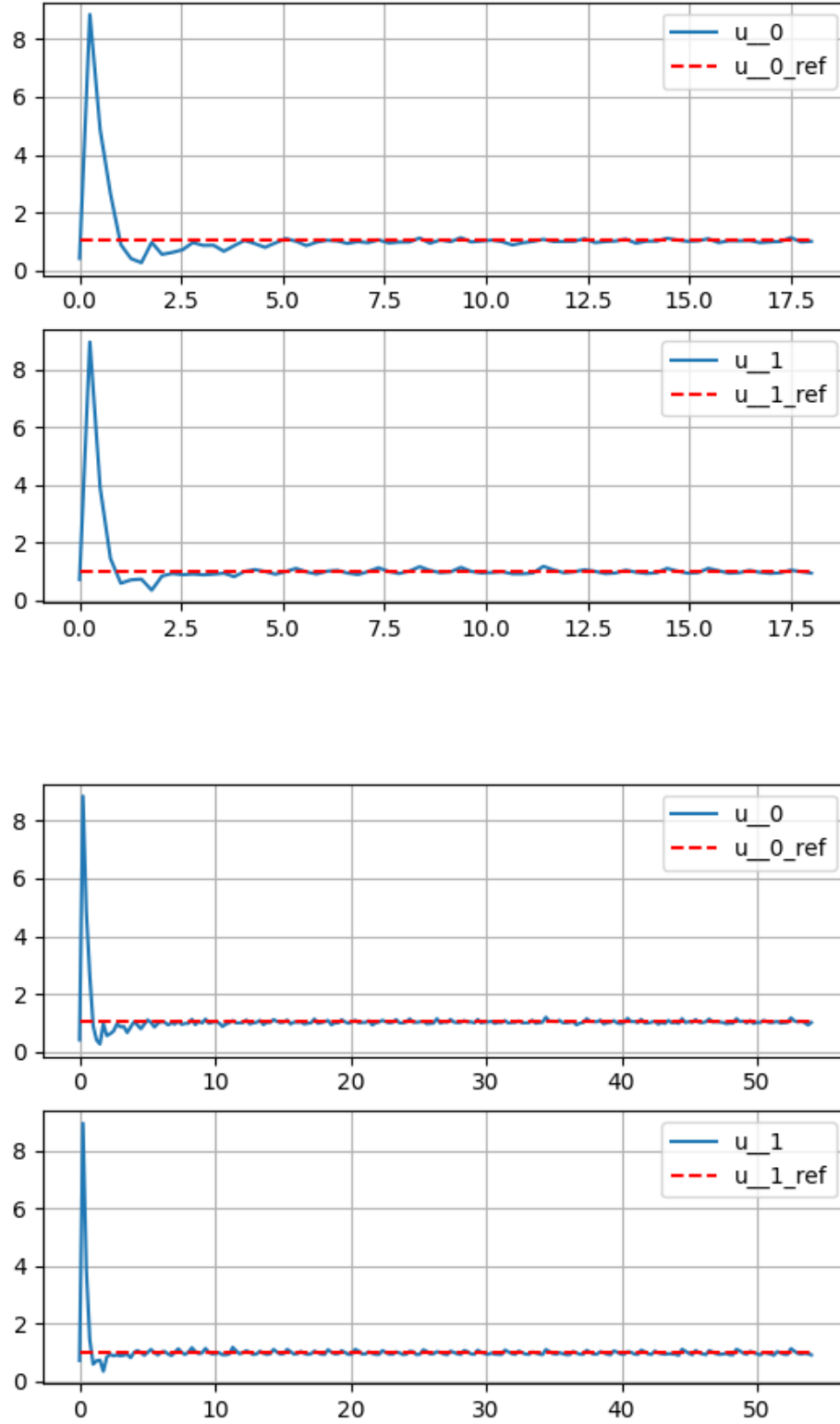


Figure 9: Intermediate input signals under robust DDMPC. Inputs adjust dynamically while converging around u_{ref} . (Insert plot series here.)

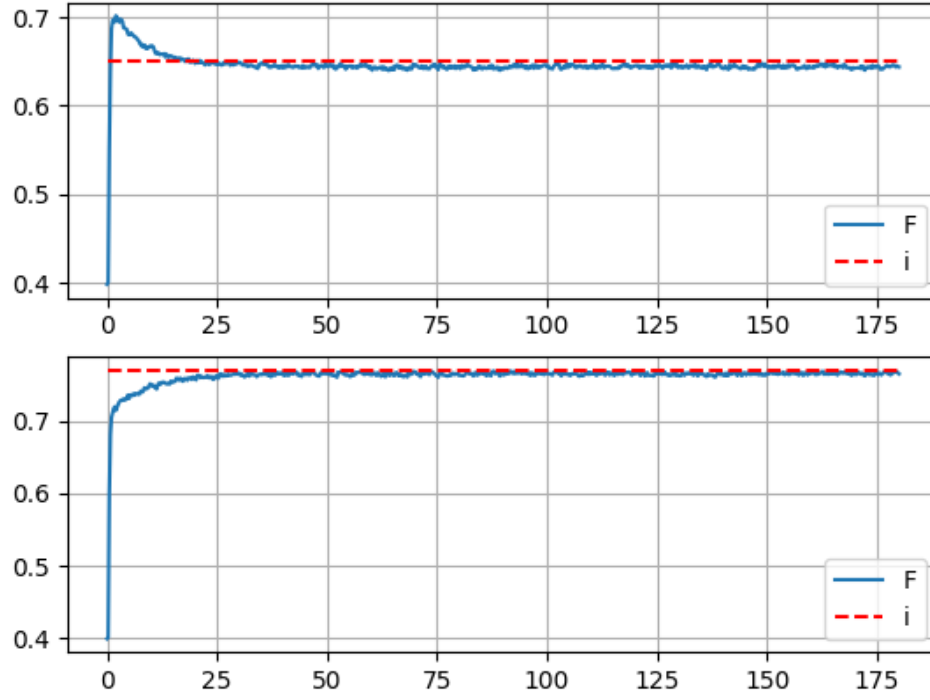


Figure 10: Closed-loop output trajectory under robust DDMPC. Outputs oscillate slightly around y_{ref} due to noise, but remain bounded.

Inputs showed similarly stable behavior, remaining within bounds and exhibiting smooth adaptation in the presence of disturbances.

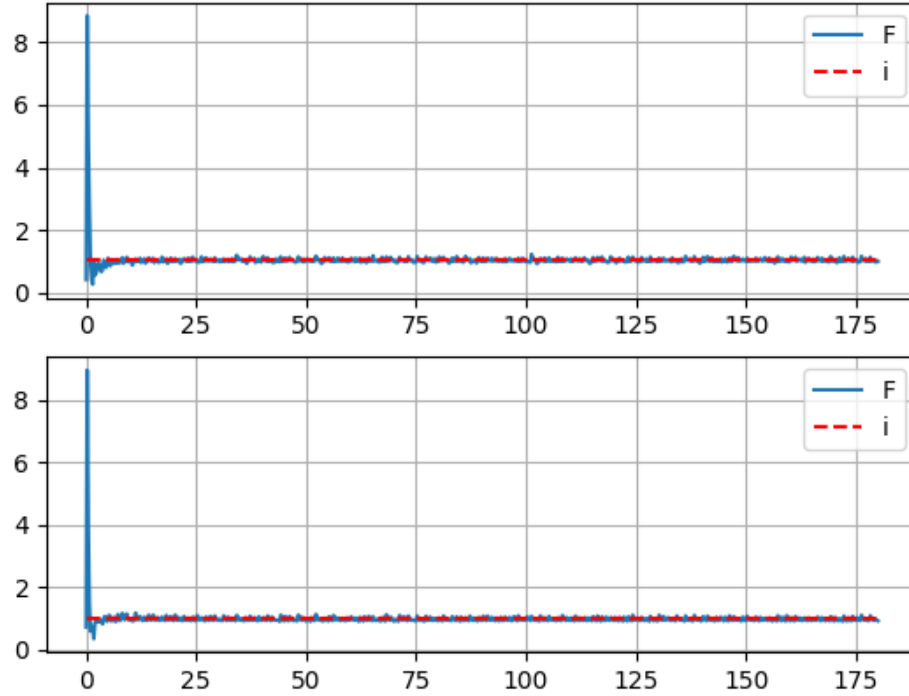
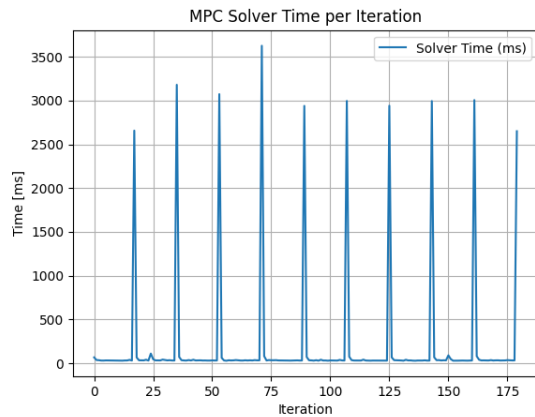


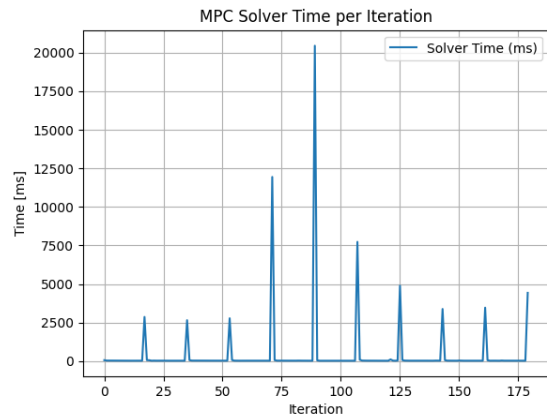
Figure 11: Control input evolution with output noise present. Regularization prevents overreactive control.

7.5 Solver Performance Analysis

Figure 12 compares the per-iteration solver times for the nominal and robust data-driven MPC schemes. Both scenarios were tested over 180 cycles using warm-started OSQP solvers.



(a) Nominal DDMPC (No Output Disturbance)



(b) Robust DDMPC (With Output Disturbance)

Figure 12: Comparison of MPC solver time per iteration. Robust DDMPC introduces higher variability and computational load due to slack variable handling and noise-related constraint complexity.

In the nominal case (Figure 12a), the majority of solver times are below 50 ms, with periodic spikes up

to 3500 ms. These spikes likely correspond to shifts in active constraints or infrequent warm-start failures. Still, the solver maintains real-time feasibility for most of the horizon.

In contrast, the robust MPC case (Figure 12b) exhibits significantly higher variability, with multiple solver spikes exceeding 10,000 ms and one peak surpassing 20,000 ms. This increase is attributed to the added complexity from:

- The introduction of the slack variable σ in the optimization,
- Noise-affected constraints that are more sensitive to feasibility margins,
- The higher condition number induced by regularization balancing $\|\alpha\|$ and $\|\sigma\|$.

These findings highlight an important trade-off: while robust DDMPC improves feasibility and tracking reliability in the presence of noise, it incurs a computational cost. In real-world applications, further tuning or solver acceleration techniques may be necessary to meet tight timing constraints.

8 Conclusion and Future Work

This internship explored the implementation, simulation, and analysis of data-driven model predictive control (DDMPC) schemes. Using only input-output data, a controller was built based on Hankel matrix representations, following the behavioral systems theory approach presented in [1, 2, 3]. Both nominal and robust DDMPC variants were implemented, tested on a linearized four-tank system, and evaluated in terms of performance, stability, and computational efficiency.

Implementation Challenges and Reflections

A central technical component of the work was the symbolic construction of the optimization problem using CasADi. While CasADi provides a powerful toolchain for modeling and solving optimal control problems, it presented specific implementation difficulties:

- Symbolic reshaping and concatenation of trajectory vectors was error-prone, especially in defining consistent indexing over input/output sequences.
- Handling warm-started variables and dual multipliers in OSQP required strict alignment between CasADi-generated constraints and solver configuration.
- CasADi’s internal memory graph and expression trees made it difficult to debug subtle shape mismatches or unintended symbolic duplications.

These challenges led to repeated refinement of the solver interface and optimization logic, ultimately deepening the understanding of both CasADi’s symbolic backend and MPC design requirements.

Behavioral Modeling vs. Explicit Dynamics

Another valuable insight came from comparing the behavioral, data-driven approach to the known physical model of the four-tank system. Since the linearized state-space model was available (via known A , B , C , D matrices), it was possible to validate whether the Hankel-based trajectory predictions aligned with true system behavior.

Empirical results indicated that:

- When the excitation data satisfied the persistency condition and noise was minimal, the Hankel-based predictor reconstructed future trajectories accurately, confirming theoretical expectations.
- Under high noise or insufficient data richness, the predictive accuracy of the behavioral model degraded noticeably compared to the known parametric model.

This provided strong justification for both the strengths and the limitations of DDMPC — it works well within the assumptions, but its success depends critically on data quality and excitation design.

Summary of Contributions

The project demonstrated the complete implementation of DDMPC in Python using CasADi, including:

- Construction of data-driven predictors using Hankel matrices.
- Implementation of nominal and robust MPC schemes with terminal constraints.
- Simulation-based validation on a multivariable system.
- Performance diagnostics including solver time profiling, convergence visualization, and trajectory accuracy.

In doing so, the internship successfully bridged theoretical methods from cutting-edge literature with hands-on control implementation and highlighted key design decisions necessary for practical deployment.

Future Work

Several directions for future investigation and extension are clear:

- **Nonlinear DDMPC:** Extend the controller to nonlinear systems using locally updated Hankel matrices and affine approximations, as suggested in [2].
- **Online data adaptation:** Implement schemes to update the trajectory data window online while preserving persistency of excitation.
- **Constraint tightening:** Apply robust constraint satisfaction techniques (e.g., tube MPC or set-invariant tightening) for bounded-noise scenarios.
- **Model-augmented DDMPC:** Explore hybrid approaches that combine data-driven predictors with partial physical models to improve prediction reliability.
- **Experimental validation:** Deploy the controller on hardware (e.g., a physical four-tank system) to evaluate real-time feasibility, sensor noise, and actuator saturation effects.

Overall, the internship provided a deep practical understanding of data-driven control, strengthened coding and optimization skills, and served as a robust introduction to the research frontier of model-free MPC.

References

- [1] J. Berberich, J. Köhler, M. A. Müller, and F. Allgöwer, “Data-Driven Model Predictive Control with Stability and Robustness Guarantees,” *IEEE Transactions on Automatic Control*, vol. 66, no. 4, pp. 1702–1717, 2021. doi:10.1109/TAC.2020.3000182
- [2] J. Berberich, J. Köhler, M. A. Müller, and F. Allgöwer, “Data-driven model predictive control: Closed-loop guarantees and experimental results,” *at – Automatisierungstechnik*, vol. 69, no. 7, pp. 608–618, 2021. doi:10.1515/auto-2021-0024
- [3] J. Berberich, J. Köhler, and F. Allgöwer, “An Overview of Systems-Theoretic Guarantees in Data-Driven Model Predictive Control,” 2023.
- [4] CasADi Documentation, <https://web.casadi.org/docs/>, Accessed: May 2025.
- [5] J. Coulson, J. Lygeros, and F. Dörfler, “Data-Enabled Predictive Control: In the Shallows of the DeePC,” *Proc. European Control Conference (ECC)*, 2019.
- [6] M. Korda and I. Mezić, “Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control,” *Automatica*, vol. 93, pp. 149–160, 2018.

- [7] L. Hewing, J. Kabzan, and M. N. Zeilinger, “Cautious Model Predictive Control Using Gaussian Process Regression,” *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2736–2743, 2020.
- [8] M. Fiedler, K. Worthmann, and P. Benner, “Recursive Data-Driven MPC with Theoretical Guarantees,” *Automatica*, vol. 155, 2023.
- [9] F. Dörfler, “Data-Enabled Predictive Control and Its Theoretical Foundations,” *Annual Reviews in Control*, vol. 55, pp. 53–70, 2023.
- [10] V. Adetola and M. Guay, “Robust adaptive MPC for constrained uncertain nonlinear systems,” *International Journal of Adaptive Control and Signal Processing*, vol. 25, no. 2, pp. 155–167, 2011.
- [11] I. Markovsky and P. Rapisarda, “Data-Driven Simulation and Control,” *International Journal of Control*, vol. 81, no. 12, pp. 1946–1959, 2008.